

IF5153 Natural Language Processing
Laporan Praktikum 6
Aspect-Based Sentiment Analysis (ABSA)

Disusun untuk memenuhi tugas mata kuliah Pemrosesan Bahasa Alami
pada Semester 1 (satu) Tahun Akademik 2025/2026
Tugas Besar IF5153 Pemrosesan Bahasa Alami



Oleh

Shabrina Maharani

13522134

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025

I. Deskripsi Persoalan

Aspect-Based Sentiment Analysis (ABSA) merupakan salah satu task dalam bidang Natural Language Processing (NLP) yang berfokus untuk menganalisis sentimen terhadap aspek-aspek spesifik dalam suatu teks, bukan hanya menilai sentimen secara kalimat keseluruhan. Dengan ABSA, sistem mampu memahami bahwa satu kalimat dapat memiliki lebih dari satu aspek dengan sentimen yang berbeda. Misalnya, pada kalimat “The pasta was delicious but the service was slow”, aspek food memiliki sentimen positif, sedangkan aspek service bersentimen negatif.

Pada praktikum ini, mahasiswa diminta untuk membangun sistem ABSA berbasis Large Language Model (LLM) dengan menerapkan berbagai teknik prompting untuk menghasilkan prediksi sentimen secara akurat terhadap setiap aspek dalam teks. Proses yang dilakukan dimulai dengan penyusunan prompt dengan memperhatikan komponen penting seperti role, task, context, format, dan constraint, lalu melakukan inference terhadap data uji menggunakan model LLM berukuran kecil ($\leq 1B$ parameter). Eksperimen dilakukan dengan minimal lima teknik prompting yang berbeda.

Tujuan akhir dari praktikum ini adalah untuk membandingkan performa berbagai teknik prompting dalam melakukan Aspect-Based Sentiment Analysis menggunakan metrik evaluasi Macro F1-Score, serta menganalisis bagaimana struktur prompt memengaruhi hasil klasifikasi sentimen terhadap setiap aspek.

II. Model yang Digunakan

Pada praktikum ini, model yang digunakan adalah google/gemma-3-1b-it, yaitu salah satu model dalam keluarga Gemma 3 yang dikembangkan oleh Google DeepMind. Gemma merupakan keluarga open-weight language model. Arsitektur yang digunakan adalah decoder-only Transformer, yang berfokus pada generative process di sisi decoder yang sangat cocok untuk tugas-tugas seperti prompt-based text generation, reasoning, maupun sentiment analysis.

Gemma 3 menggunakan arsitektur Transformer decoder-only, artinya seluruh proses pemrosesan bahasa dilakukan melalui blok decoder tanpa menggunakan encoder. Mekanisme utamanya

adalah self-attention, yang membuat model dapat memahami hubungan antar token dalam konteks yang panjang.

Secara sederhana, proses kerjanya dapat adalah sebagai berikut:

1. Tokenisasi

Teks input dipecah menjadi token-token menggunakan tokenizer bawaan. Setiap token kemudian diubah menjadi vektor numerik dengan menggunakan embedding layer.

2. Pemrosesan dengan Self-Attention

Setiap token diproses secara paralel dengan memperhatikan konteks token lain melalui mekanisme multi-head self-attention. Dengan cara ini, model dapat memahami makna kata berdasarkan konteks di sekitarnya.

3. Prediksi Token Selanjutnya

Karena bersifat causal (autoregresif), model memprediksi token berikutnya berdasarkan semua token sebelumnya. Proses ini berulang hingga mencapai jumlah token yang diinginkan atau hingga model menghasilkan end of sequence <eos>.

4. Text Generation

Seluruh token yang dihasilkan digabung kembali menjadi teks jawaban sebagai hasil keluaran model.

Gemma 3 sendiri memiliki dua jenis utama, yaitu:

- gemma-3-1b-pt (Pretrained Model) yaitu versi dasar yang hanya melalui tahap pretraining yang digunakan untuk aplikasi yang memerlukan fine-tuning sesuai kebutuhan spesifik pengguna.
- gemma-3-1b-it (Instruction Tuned Model) yaitu model yang telah melalui tahap instruction tuning agar dapat merespons instruksi manusia secara natural dan kontekstual sehingga sangat sesuai untuk eksperimen prompt engineering.

Selain itu, model Gemma ini memiliki context window sebanyak 32.768 token dan dapat digunakan dengan lebih dari 140 bahasa termasuk bahasa Indonesia, serta optimal digunakan pada GPU maupun TPU dengan resource rendah.

Gemma-3-1b-it seperti yang sebelumnya dijelaskan merupakan model yang telah melalui tahap *instruction tuning*. Jadi, model ini merupakan pengembangan lanjutan dari model *base* (gemma-3-1b-pt) yang dilatih ulang dengan data instruksi dengan berbagai ragam instruksi untuk meningkatkan kemampuannya dalam memahami dan merespons perintah atau prompt yang dibuat pengguna.

Tahap *instruction tuning* terdiri dari dua proses penting yaitu,

- Supervised Fine-Tuning (SFT) dimana model dilatih menggunakan pasangan data *prompt-response* sehingga sapat menjawab instruksi dengan cara yang relevan dan koheren.
- Reinforcement Learning from Human Feedback (RLHF) dimana model disesuaikan lebih lanjut berdasarkan preferensi manusia agar hasilnya terdengar lebih natural dan sesuai konteks percakapan.

Dari tahapan tersebut gemma-3-1b-it jadi dapat memahami perintah seperti "*Klasifikasikan sentimen aspek berikut*" atau "*Tuliskan jawaban dalam satu kata*", tanpa perlu penyesuaian tambahan.

Model ini dipilih karena berukuran 1 miliar parameter ($\leq 1B$), sesuai dengan batas yang ditentukan dalam spesifikasi praktikum, serta telah melalui proses *instruction tuning* sehingga mampu memahami dan menanggapi *prompt* dengan baik. Meskipun berukuran hanya 1 miliar parameter, model ini dirancang agar tetap efisien dan kompetitif. Beberapa alasan lainnya terkait pemilihan model ini adalah

- Multilingual Understanding dimana model ini mendukung lebih dari 140 bahasa, termasuk bahasa Indonesia.
- Context Length yang membuat model dapat memproses konteks hingga 32.768 token sehingga dapat membuat model mampu menganalisis teks panjang dengan konsistensi konteks yang baik.
- Optimization for Efficiency yang membuat model dapat berjalan secara optimal pada GPU dan TPU dengan memanfaatkan teknik kompresi bobot dan mixed-precision

training sehingga dapat mendukung inferensi dalam waktu yang cepat dengan konsumsi sumber daya minimal.

- Safety & Bias Mitigation yang membuat model Gemma 3 dilengkapi dengan filter dan penyaringan untuk mengurangi hasil output yang berpotensi bias atau berbahaya.

Model gemma-3-1b-it bekerja dengan memahami pola instruksi yang diberikan melalui prompt, misalnya:

Sentence: "The pasta was delicious but the service was slow." Aspect: "service" Answer:

Dengan adanya instruction tuning yang dimiliki, Gemma bisa memahami bahwa instruksi yang diberikan bukan untuk menilai seluruh kalimat, melainkan sentimen terhadap aspek tertentu, misalnya “service”. Jadi, ketika diberikan kalimat seperti “The pasta was delicious but the service was slow” dengan aspek “service”, model ini akan tahu bahwa fokusnya ada pada layanan, lalu menghasilkan jawaban seperti “negative” sesuai makna kalimat tersebut.

Selain itu, ketika digunakan bersama berbagai teknik prompting seperti Zero-Shot, Few-Shot, atau Chain-of-Thought, performa Gemma bisa berbeda-beda. Model ini mampu menyesuaikan cara berpikirnya sesuai struktur prompt yang diberikan, memahami konteks instruksi dengan lebih baik, serta memberikan label sentimen yang lebih akurat dan konsisten.

1. Proses dalam Notebook

Sebelum melakukan proses prompting dan inference, tahap awal yang dilakukan pada notebook adalah autentikasi akun Hugging Face. Proses ini diperlukan agar notebook dapat mengakses model yang disimpan di repositori Hugging Face. Langkahnya dimulai dengan login ke situs Hugging Face menggunakan akun pribadi, kemudian membuat access token melalui menu profil (biasanya di bagian Settings → Access Tokens). Setelah token dibuat, akan dijalankan cell awal perintah `!huggingface-cli login` di dalam notebook, lalu pengguna dapat copy-paste token tersebut ke terminal dalam notebook. Jika proses autentikasi berhasil, maka notebook sudah terhubung dengan akun Hugging Face dan bisa memuat model yang diinginkan.

Selanjutnya, model `google/gemma-3-1b-it` di-load menggunakan fungsi `AutoModelForCausalLM.from_pretrained()` yang disertai `AutoTokenizer.from_pretrained()`. Fungsi `AutoTokenizer` digunakan untuk mengubah teks menjadi token numerik yang bisa diproses oleh model, sedangkan `AutoModelForCausalLM` berfungsi memanggil model berbasis Causal Language Modeling (decoder-only) yang digunakan untuk menghasilkan teks secara autoregresif yaitu memprediksi token berikutnya berdasarkan konteks token sebelumnya. Model kemudian di-wrap menggunakan pipeline `text-generation` dari Hugging Face, yang mempermudah proses generasi teks dari prompt tanpa perlu menulis kode inferensi secara manual.

Setelah model berhasil di-load, dibuat fungsi wrapper `generate_text()` untuk menyederhanakan pemanggilan model selama inference. Fungsi ini digunakan untuk menghasilkan teks berdasarkan prompt yang diberikan dengan mode deterministik (`do_sample=False`). Setelah itu, ditambahkan juga proses normalization melalui fungsi `normalize_label()`, yang bertugas untuk menyamakan berbagai bentuk output model menjadi label yang seragam seperti “positive”, “negative”, “neutral”, atau “conflict”. Misalnya, output seperti “positif” atau “pos” akan otomatis dikonversi menjadi “positive”.

Selanjutnya, dilakukan proses inference untuk setiap varian prompt yang telah disiapkan di variabel `PROMPT_VARIANTS`. Setiap teknik prompting (misalnya Zero-Shot, Few-Shot, atau Self-Consistency) dijalankan untuk memprediksi sentimen berdasarkan pasangan kalimat dan aspek. Untuk varian Self-Consistency, model dijalankan beberapa kali dengan pengaturan random sample menggunakan parameter seperti `temperature`, `top_p`, dan `top_k` untuk menghasilkan beberapa prediksi yang kemudian digabungkan menggunakan metode voting.

Hasil dari setiap teknik prompting kemudian disimpan pada dataframe untuk dianalisis lebih lanjut. Pada tahap akhir, karena model Gemma 3 bersifat decoder-only dan sudah melalui tahap pretraining serta instruction tuning, maka evaluasi performanya tidak dilakukan menggunakan validation set seperti teknik tradisional. Hasil prediksi yang dihasilkan dievaluasi melalui hasil

submisi pada kompetisi Kaggle dengan menggunakan metrik Macro F1-Score untuk menilai seberapa baik model mengenali sentimen dari setiap aspek yang diuji.

2. Penjelasan Parameter dan Konfigurasi

Dalam proses inferensi, beberapa parameter digunakan untuk mengatur behaviour dari generative diantaranya adalah

- **temperature**

Parameter ini mengatur seberapa “acak” hasil teks yang dibuat oleh model. Kalau nilainya tinggi (seperti 1.0), hasilnya akan lebih beragam dan kreatif, sedangkan kalau nilainya rendah (sekitar 0.2–0.5), hasilnya cenderung lebih pasti dan tidak banyak variasi. Pada praktikum ini, nilai temperature = 1.0 digunakan supaya model bisa menghasilkan beberapa jawaban yang berbeda ketika menjalankan metode Self-Consistency.

- **top_p (Nucleus Sampling)**

Parameter ini menentukan seberapa besar kemungkinan kata-kata yang akan dipilih oleh model. Pada praktikum ini, top_p = 0.95, artinya model hanya akan mempertimbangkan kumpulan kata dengan total peluang tertinggi sampai 95%. Cara ini membantu model tetap menghasilkan teks yang masuk akal tapi tetap punya variasi.

- **top_k**

Parameter ini membatasi berapa banyak kata dengan peluang tertinggi yang bisa dipilih model saat membuat teks. Nilai top_k = 64 digunakan agar pemilihan token tetap beragam namun masih dalam ruang lingkup yang relevan.

- **do_sample**

Saat diatur ke True, model akan melakukan *sampling* acak berdasarkan distribusi probabilitas token, bukan hanya memilih token dengan probabilitas tertinggi (*greedy decoding*). Pada metode *Self-Consistency*, pengaturan ini digunakan untuk menghasilkan beberapa sampel berbeda yang kemudian dibandingkan melalui proses *voting*.

- **max_new_tokens**

Parameter ini menentukan jumlah token maksimum yang dihasilkan model untuk setiap *prompt*. Dalam eksperimen ini, nilainya dibatasi hingga 32 token agar hasil tetap ringkas dan efisien.

III. Analisis Prompt

1. Penjelasan Jenis Prompt

Sepuluh prompt yang digunakan pada praktikum ini untuk dianalisis dalam menyelesaikan ABSA. Berikut adalah penjelasan dari sepuluh prompt tersebut.

1. Zero Shot Prompting

```
PROMPT_ZERO = (  
    "Classify the sentiment towards the given aspect in the sentence below.\n"  
    "Answer with ONE WORD only from: positive, negative, neutral, conflict."  
)
```

Pada Zero-Shot Prompting, model diberi instruksi langsung tanpa contoh atau panduan tambahan. Pendekatan ini pertama kali diperkenalkan oleh Brown et al. (2020) dalam paper “Language Models are Few-Shot Learners”. Konsep utamanya adalah LLM dapat memahami dan menyelesaikan tugas baru hanya dengan membaca instruksi dalam natural language, tanpa pelatihan ulang atau pemberian contoh. Secara arsitektural, Gemma-3-1b-it sebagai decoder-only transformer akan menafsirkan kalimat perintah sebagai konteks untuk menghasilkan token berikut yang sesuai dengan instruksi. Karena tidak ada contoh atau panduan tambahan, model bergantung sepenuhnya pada world knowledge yang telah ia pelajari selama tahap pretraining untuk menentukan label sentimen yang paling mungkin. Berikut adalah elemen-element prompt yang ada pada zero shot prompting.

- Instruction: “Classify the sentiment towards the given aspect in the sentence below.”
- Context: Tidak ada konteks tambahan.
- Input Data: Kalimat dan aspek yang dinilai.
- Output Indicator: “Answer with ONE WORD only from: positive, negative, neutral, conflict.”

2. Few Shot Prompting

```
PROMPT_FEW = (  
    "You are an ABSA assistant. Decide the sentiment towards the given aspect.\n"  
    "Answer with ONE WORD only from: positive, negative, neutral, conflict.\n\n"  
    "Examples:\n"  
    "Sentence: \"The sushi was fresh and tasty.\" Aspect: \"sushi\" Answer:  
positive\n"  
    "Sentence: \"The soup arrived cold and was bland.\" Aspect: \"soup\" Answer:  
negative\n"  
    "Sentence: \"The decor is fine.\" Aspect: \"decor\" Answer: neutral\n"  
    "Sentence: \"The coffee is great sometimes, but often terrible.\" Aspect:  
\"coffee\" Answer: conflict\n"  
    "Now classify the following pair."  
)
```

Dalam Few-Shot Prompting, model tidak hanya diberi instruksi, tetapi juga beberapa contoh (demonstrations) yang menunjukkan hubungan antara kalimat, aspek, dan label. Konsep ini juga dijelaskan oleh Brown et al. (2020) sebagai bagian dari fenomena In-Context Learning, di mana model dapat mempelajari pola baru hanya dari contoh yang disajikan di dalam prompt, tanpa pelatihan ulang.

Dari sisi arsitektur, model Gemma-3-1b-it membaca seluruh prompt sebagai satu urutan teks. Dengan mekanisme self-attention, model mengenali pola dari contoh yang diberikan (misalnya hubungan kata “fresh” dengan label positive), lalu mengaplikasikan pola itu untuk memprediksi label pada kasus baru. Berikut adalah elemen-elemen prompt yang ada pada few shot prompting.

- Instruction: “You are an ABSA assistant. Decide the sentiment towards the given aspect.”
- Context: Contoh kalimat dan label yang berfungsi sebagai panduan pola. (“Examples ...”)
- Input Data: Kalimat dan aspek baru yang belum diklasifikasikan. (“Sentence ...”)
- Output Indicator: “Answer with ONE WORD only from: positive, negative, neutral, conflict.”

3. Role Prompting

```
PROMPT_ROLE = (  
    "You are a meticulous and highly critical restaurant reviewer. "  
    "Your task is to classify the sentiment (positive, negative, neutral,  
conflict) "  
)
```

```
"towards the specific aspect provided. "  
"You must answer with ONE WORD only and nothing else."  
)
```

Role Prompting didasarkan pada konsep role-play prompting yang dijelaskan oleh Shanahan, McDonell, & Reynolds (2023) dalam paper “Role-Play with Large Language Models.” Dalam pendekatan ini, model diberikan sebuah role atau peran tertentu agar dapat menjawab dengan cara berpikir dan gaya bahasa yang meniru karakter peran tersebut. Misalnya, dengan memberikan peran “kritikus restoran yang teliti dan kritis”, model diarahkan untuk lebih peka terhadap nuansa positif dan negatif pada aspek makanan, pelayanan, maupun dekorasi.

Secara arsitektural, bagian awal prompt yang menyebutkan “You are a meticulous and highly critical restaurant reviewer” berfungsi sebagai context embedding tambahan yang memengaruhi cara model menghasilkan urutan token. Decoder-only transformer seperti Gemma-3-1B-IT akan memperlakukan deskripsi peran itu sebagai konteks semantik yang membentuk bias dalam prediksi token berikutnya, sehingga jawaban yang dihasilkan mencerminkan karakter peran yang diminta. Berikut adalah elemen-elemen prompt yang ada pada role prompting.

- Instruction: “You are a meticulous and highly critical restaurant reviewer.”
- Context: Penjelasan tambahan tentang tugas yang dilakukan, yaitu mengklasifikasikan sentimen terhadap aspek tertentu.
- Input Data: Kalimat dan aspek yang akan dinilai.
- Output Indicator: “You must answer with ONE WORD only and nothing else.”

4. Step Back Prompting

```
PROMPT_STEP_BACK = (  
    "You are an ABSA assistant. Before answering, take a step back and state  
    the general principle "  
    "for classifying sentiment for the aspect based on the sentence. "  
    "Then, apply this principle to give the final answer.\n"  
    "Output the final label as ONE WORD only: positive, negative, neutral, or  
    conflict.\n"  
    "Final answer (one word only):"  
)
```

Step-Back Prompting diperkenalkan oleh Zheng et al. (2023) dalam paper “Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models.” Teknik ini mendorong model untuk melakukan penalaran tingkat abstrak dengan “mundur satu langkah” sebelum menjawab. Model diminta menuliskan prinsip umum terlebih dahulu, baru kemudian memberikan label akhir.

Arsitekturnya memanfaatkan kemampuan multi-token reasoning dari LLM. Saat membaca perintah “take a step back,” model membangun representasi internal berupa konsep umum (misalnya “kalimat positif biasanya berisi pujian”), lalu menerapkannya untuk menentukan label yang sesuai. Berikut adalah elemen-element prompt yang ada pada step back prompting.

- Instruction: “Before answering, take a step back and state the general principle...”
- Context: Permintaan agar model menjelaskan prinsip sebelum menjawab.
- Input Data: Kalimat dan aspek yang ingin dinilai.
- Output Indicator: “Output the final label as ONE WORD only.”

5. Zero Shot + Chain of Thought Prompting

```
PROMPT_ZERO_COT = (  
    "You are an ABSA assistant. Classify the sentiment towards the given  
    aspect.\n"  
    "Let's think step by step."  
)
```

Zero-Shot CoT dikembangkan oleh Kojima et al. (2022) dalam paper “Large Language Models are Zero-Shot Reasoners.” Mereka menemukan bahwa hanya dengan menambahkan frasa sederhana seperti “Let’s think step by step”, model dapat menalar secara bertahap sebelum memberikan jawaban akhir. Teknik ini memperluas kemampuan penalaran model tanpa contoh tambahan.

Secara arsitektural, frasa tersebut berfungsi sebagai trigger token yang memicu model untuk menuliskan proses berpikirnya dulu sebelum menjawab, sehingga hasil akhirnya lebih masuk akal dan tidak asal menebak. Model secara internal “membangun alasan” sebelum sampai pada kata akhir, sehingga hasilnya lebih logis. Berikut adalah elemen-element prompt yang ada pada prompting ini.

- Instruction: “Classify the sentiment towards the given aspect.”
- Context: Frasa “Let’s think step by step.”
- Input Data: Kalimat dan aspek target.
- Output Indicator: Jawaban diharapkan berupa satu kata label sentimen.

6. Few Shot + Chain of Thought Prompting

```
PROMPT_FEW_COT = (
    "You are an ABSA assistant. Think step by step before deciding the label,
    then output the final label as ONE WORD.\n\n"
    "--- Example 1 ---\n"
    "Sentence: \"The sushi was fresh and tasty.\" Aspect: \"sushi\"\n"
    "Steps:\n"
    "1) The opinion phrases related to 'sushi' are 'fresh' and 'tasty'.\n"
    "2) Both 'fresh' and 'tasty' are positive.\n"
    "3) The final label is positive.\n"
    "Final answer (one word only): positive\n\n"
    "--- Example 2 ---\n"
    "Sentence: \"The soup arrived cold and was bland.\" Aspect: \"soup\"\n"
    "Steps:\n"
    "1) The opinion phrases related to 'soup' are 'arrived cold' and
    'bland'.\n"
    "2) Both 'arrived cold' and 'bland' are negative.\n"
    "3) The final label is negative.\n"
    "Final answer (one word only): negative\n\n"
    "--- Example 3 ---\n"
    "Sentence: \"The coffee is great sometimes, but often terrible.\" Aspect:
    \"coffee\"\n"
    "Steps:\n"
    "1) The opinion phrases related to 'coffee' are 'great' and 'terrible'.\n"
    "2) 'great' is positive, but 'terrible' is negative. This is a
    conflict.\n"
    "3) The final label is conflict.\n"
    "Final answer (one word only): conflict\n\n"
    "--- Task ---\n"
)
```

Few-Shot CoT diperkenalkan oleh Wei et al. (2022) dalam paper “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” Konsepnya adalah memberi contoh penalaran step by step agar model dapat meniru proses berpikir sebelum menjawab. Pendekatan ini meningkatkan kemampuan logika dan interpretasi konteks model.

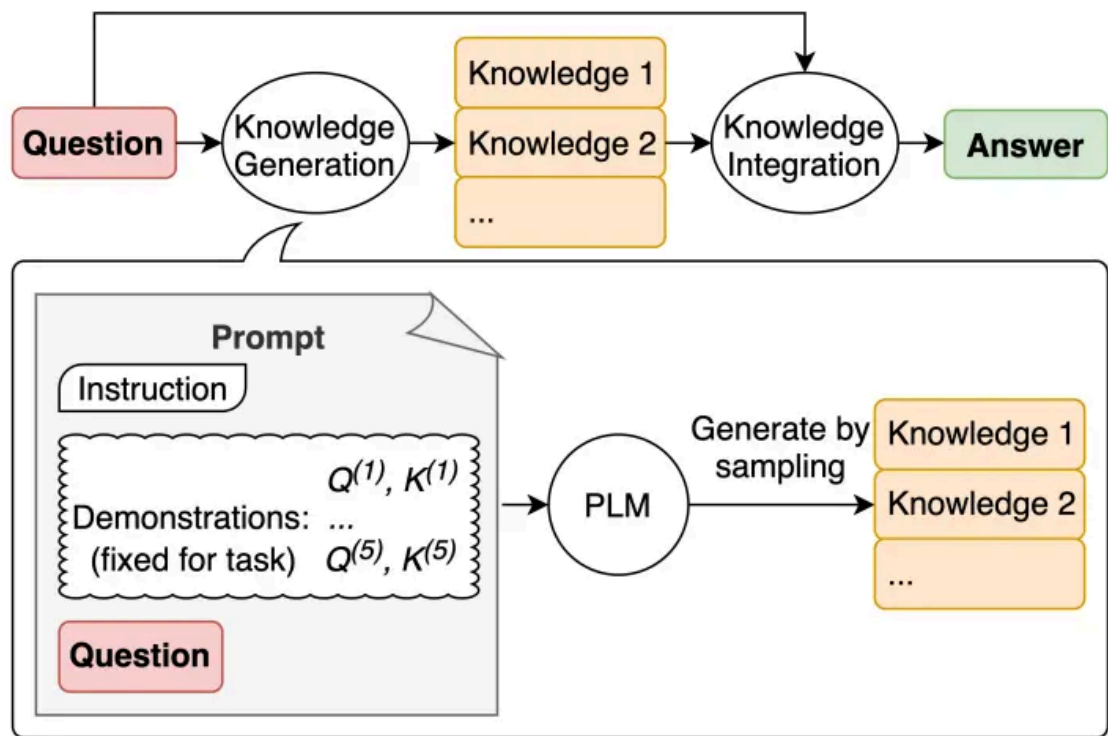
Secara arsitektural, model membaca contoh yang terdiri dari sentence, steps, dan final answer sebagai urutan token. Dengan mekanisme attention, model belajar bahwa bagian “Steps” berisi proses berpikir, lalu menerapkan untuk kasus baru. Berikut adalah elemen-element prompt yang ada pada prompting ini.

- **Instruction:** “Think step by step before deciding the label.”
- **Context:** Contoh lengkap berisi langkah-langkah penalaran.
- **Input Data:** Kalimat dan aspek baru.
- **Output Indicator:** “Output the final label as ONE WORD.”

7. Generative Knowledge Prompting

```
PROMPT_GENERATE_KNOWLEDGE = (
    "You are an ABSA assistant. First, generate one sentence of relevant
    knowledge about the aspect. "
    "Then, classify the sentiment (positive, negative, neutral, conflict)
    based on that knowledge. "
    "Answer with the sentiment label only.\n\n"
    "--- Example 1 ---\n"
    "Sentence: \"The soup arrived cold and was bland.\" Aspect: \"soup\"\n"
    "Knowledge: Soup is generally expected to be served hot and be flavorful;
    'cold' and 'bland' are undesirable qualities.\n"
    "Answer: negative\n\n"
    "--- Example 2 ---\n"
    "Sentence: \"The decor is fine.\" Aspect: \"decor\"\n"
    "Knowledge: The word 'fine' in a review context typically indicates
    mediocrity, not strong positive or negative feelings.\n"
    "Answer: neutral\n\n"
    "--- Task ---\n"
)
```

Teknik ini diusulkan oleh Liu et al. (2021) dalam “Generated Knowledge Prompting for Commonsense Reasoning.” Model diarahkan untuk terlebih dahulu menghasilkan pengetahuan relevan (knowledge generation), kemudian menggunakan pengetahuan itu untuk menentukan jawaban (knowledge integration). Pendekatan dua tahap ini membantu model ketika konteks kalimat tidak cukup jelas.



Sumber Gambar : <https://arxiv.org/pdf/2110.08387>

Arsitekturnya menyerupai dua tahap proses seperti diagram:

1. Question → Knowledge Generation → Knowledge Integration → Answer.
2. PLM (dalam hal ini Gemma-3) melakukan sampling untuk menghasilkan potongan pengetahuan, lalu menggunakannya untuk memprediksi label.

Berikut adalah elemen-element prompt yang ada pada generative knowledge prompting.

- Instruction: “First, generate one sentence of relevant knowledge...”
- Context: Dua contoh yang memperlihatkan hubungan antara knowledge dan answer.
- Input Data: Kalimat dan aspek.
- Output Indicator: “Answer with the sentiment label only.”

8. Self-Consistency

```
PROMPT_SELF_CONSISTENCY = (  
    "You are an ABSA assistant. Think step by step before deciding the  
    label.\n"  
    "Steps:\n"  
    "1) Extract opinion phrases related to the aspect.\n"  
    "2) Determine polarity towards that aspect.\n"  
    "3) Output the final label as ONE WORD from: positive, negative, neutral,  
    conflict.\n"  
    "Final answer (one word only):"  
)
```

Self-Consistency diperkenalkan oleh Wang et al. (2022) dalam “Self-Consistency Improves Chain-of-Thought Reasoning in Language Models.” Teknik ini menjalankan prompt yang sama beberapa kali dengan beberapa hasil random yang berbeda (sampling), lalu menentukan hasil akhir berdasarkan voting mayoritas. Tujuannya untuk mengurangi bias penalaran jika hanya satu jawaban.

Dari sisi arsitektur, model menjalankan beberapa forward pass dengan nilai random berbeda (misalnya pada temperature). Semua hasil disimpan, lalu dihitung label yang paling sering muncul. Berikut adalah elemen-elemen prompt yang ada pada self-consistency prompting.

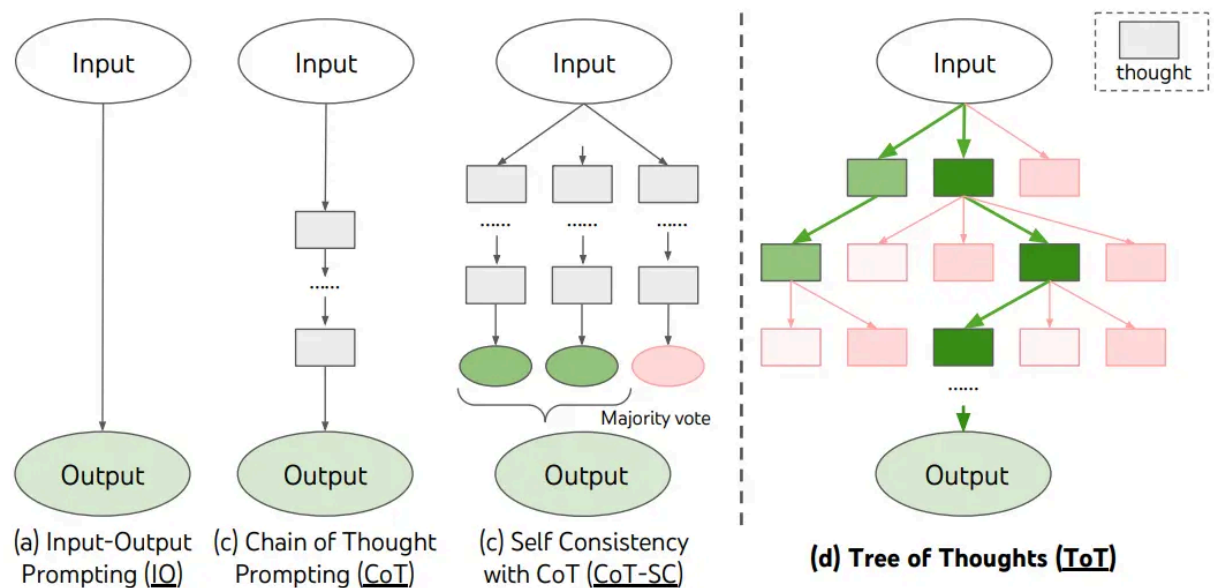
- Instruction: “Think step by step before deciding the label.”
- Context: Petunjuk langkah-langkah penalaran.
- Input Data: Kalimat dan aspek.
- Output Indicator: “Output the final label as ONE WORD from: positive, negative, neutral, conflict.”

9. Tree of Thought (ToT)

```
PROMPT_TOT_SIMULATED = (  
    "You are an ABSA assistant using a Tree of Thought process.\n"  
    "Task: Determine the sentiment for the given aspect.\n\n"  
    "Step 1: Generate 3 distinct reasoning paths (branches) to evaluate the  
    sentiment.\n"  
    "Step 2: Evaluate each branch. State which branch is the most logical.\n"  
    "Step 3: Based on the best branch, conclude the final sentiment.\n"  
    "Output the final label as ONE WORD from: positive, negative, neutral,  
    conflict.\n"  
    "Final answer (one word only):"  
)
```

Tree-of-Thought (ToT) Prompting merupakan pengembangan dari Chain-of-Thought (CoT), pertama kali diperkenalkan oleh Yao et al. (2023) dalam paper berjudul “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.”

Pendekatan ini dirancang untuk menyelesaikan tugas-tugas kompleks yang membutuhkan eksplorasi, penilaian, dan lookahead reasoning yang menjadi hal yang sulit dicapai oleh prompting biasa seperti Zero-Shot atau CoT yang bersifat linear.



Sumber Gambar : <https://arxiv.org/pdf/2305.10601>

Secara konseptual, ToT memperluas CoT menjadi struktur bercabang. Jika CoT hanya menghasilkan satu alur berpikir berurutan, ToT memungkinkan model untuk:

1. Menghasilkan beberapa thoughts atau langkah penalaran alternatif (cabang-cabang pohon),
2. Mengevaluasi setiap cabang berdasarkan logika dan kelayakannya,
3. Memilih jalur terbaik yang mengarah pada solusi paling masuk akal.

Setiap thought dianggap sebagai urutan kalimat dan berfungsi sebagai langkah perantara menuju solusi akhir. Dengan cara ini, model dapat melakukan lookahead (melihat ke

depan beberapa langkah), serta backtracking (kembali ke jalur sebelumnya jika hasilnya tidak memadai).

Menurut Yao et al. (2023), proses ini dapat dioptimalkan menggunakan algoritma pencarian seperti Breadth-First Search (BFS), Depth-First Search (DFS), atau Beam Search untuk menelusuri dan mengevaluasi berbagai jalur penalaran. Dalam praktikum ini, implementasi ToT dilakukan secara simulatif: model diminta untuk membuat beberapa cabang penalaran, mengevaluasi masing-masing, lalu memilih hasil terbaik tanpa benar-benar menerapkan algoritma pencarian BFS/DFS. Berikut adalah elemen-element prompt yang ada pada ToT prompting.

- Instruction: “You are an ABSA assistant using a Tree of Thought process.”
- Context: Tiga langkah eksplorasi, evaluasi, dan pemilihan hasil terbaik.
- Input Data: Kalimat dan aspek.
- Output Indicator: “Output the final label as ONE WORD...”

10. Emotion Prompting

```
PROMPT_EMOTION = (  
    "Classify the sentiment towards the given aspect. This is very important  
    for my analysis.\n"  
    "Answer with ONE WORD only from: positive, negative, neutral, conflict."  
)
```

Emotion Prompting diperkenalkan oleh Li et al. (2023) dalam “Large Language Models Understand and Can Be Enhanced by Emotional Stimuli.” Penelitian mereka menunjukkan bahwa ekspresi emosional di dalam instruksi dapat memengaruhi tingkat kepatuhan dan fokus model. Frasa seperti “This is very important for my analysis” membuat model memperlakukan tugas dengan “serius”, menghasilkan jawaban yang lebih konsisten.

Secara arsitektur, token emosional seperti “important” mengubah distribusi perhatian (attention weights) dalam transformer, menyebabkan model memperkuat fokus pada konteks tugas. Berikut adalah elemen-element prompt yang ada pada emotion prompting.

- Instruction: “Classify the sentiment towards the given aspect.”

- Context: Frasa emosional “This is very important for my analysis.”
- Input Data: Kalimat dan aspek.
- Output Indicator: “Answer with ONE WORD only from: positive, negative, neutral, conflict.”

Secara keseluruhan, perbedaan utama antar sepuluh jenis prompting terletak pada cara model diarahkan untuk menalar dan menghasilkan jawaban.

2. Analisis Hasil Eksperimen

Evaluasi dilakukan menggunakan metrik Macro F1-score. Karena eksperimen ini berbasis prompting (tanpa fine-tuning), maka tidak digunakan skema train-validation split. Selain itu, data uji (test.csv) bersifat blind test di kompetisi Kaggle sehingga label aslinya tidak tersedia. Oleh karena itu, nilai Macro F1-score diperoleh melalui hasil submission ke Kaggle. Hasil lengkap evaluasi ditunjukkan pada tabel berikut.

Tabel 1. Peringkat Macro F1 Score Hasil Prompt

Peringkat	Prompt	Macro AVG F1-Score
1	Role Prompting	0.56439
2	Generative Knowledge Prompting	0.50311
3	Zero Shot Prompting	0.46097
4	Emotion Prompting	0.45873
5	Few Shot Prompting	0.44976
6	Few Shot + Chain of Thought	0.43795
7	Tree of Thought (ToT) Prompting	0.42488
8	Step Back Prompting	0.41279
9	Self-Consistency Prompting	0.40791
10	Zero Shot + Chain of Thought Prompting	0.40306

Selain Macro F1-score, berikut adalah distribusi jumlah per label yang dapat digunakan juga dalam analisis hasil eksperimen.

Label counts per variant:				
	positive	negative	neutral	conflict
variant				
zero	678	262	60	0
few	717	128	101	54
role	569	327	100	4
step_back	736	206	30	28
zero_cot	796	181	23	0
few_cot	380	115	501	4
gen_knowledge	536	217	245	2
self_consistency	734	251	4	11
tot_simulated	670	250	38	42
emotion	685	262	53	0

Gambar 1. Distribusi Jumlah per Label

Berdasarkan hasil evaluasi menggunakan Macro F1-Score dari submission Kaggle dan analisis distribusi label pada tiap teknik prompting, eksperimen ini menunjukkan hasil bahwa teknik yang sederhana dan lebih berfokus pada konteks memberikan hasil yang lebih baik dibandingkan teknik yang melibatkan proses penalaran yang rumit. Model Gemma-3-1b-it terlihat lebih konsisten ketika diberikan prompt yang bersifat langsung dan jelas (instructional dan contextual), dibandingkan teknik dengan konsep penalaran yang lebih kompleks karena mengharuskan adanya proses berpikir secara bertahap seperti Chain of Thought (CoT), Self-Consistency, atau Tree of Thought (ToT).

Teknik seperti CoT, Self-Consistency, dan ToT disebut “kompleks secara penalaran” karena tidak hanya meminta model menghasilkan satu jawaban akhir, tetapi juga meminta model untuk meniru proses berpikir bertahap yang menyerupai penalaran manusia. Terdapat beberapa hal yang menjadi faktor keberhasilan atau kegagalan sebuah teknik prompt memprediksi kelas sentimen.

Faktor pertama yang dapat menyebabkan lima varian prompt (Few-Shot CoT, Zero-Shot CoT, Step-Back, ToT, dan Self-Consistency) dengan skor terendah adalah teknik yang paling kompleks secara instruksional serta adanya keterbatasan penalaran model (the curse of complexity). Pola ini mengindikasikan bahwa model Gemma-3-1B-IT, dengan kapasitas sekitar 1 miliar parameter, belum cukup kuat untuk mengeksekusi instruksi yang mengharuskan adanya penalaran secara bertahap. Model decoder-only seperti Gemma pada dasarnya bekerja sebagai “mesin pelengkap teks”. Model ini akan memprediksi token berikutnya berdasarkan konteks sebelumnya. Prompt yang sederhana hanya membutuhkan satu token jawaban (“positive”), sementara prompt kompleks yang membutuhkan jawaban penalaran memaksa model untuk menghasilkan serangkaian token yang membentuk logika, evaluasi, atau argumen. Berikut adalah tiga model dengan penalaran yang kompleks yang menghasilkan nilai macro F1-score yang berada di kategori 5 peringkat terendah.

- Chain of Thought (CoT) yang mengharuskan model menjelaskan langkah-langkah penalaran sebelum memberikan label akhir (“Let’s think step by step”).
- Self-Consistency yang menjalankan CoT beberapa kali dan melakukan voting untuk mencari jawaban paling konsisten (pada eksperimen ini, N_SAMPLES=5).
- Tree of Thought (ToT) yang menyimulasikan penalaran bercabang dan membuat model diminta menghasilkan beberapa alternatif jalur berpikir, mengevaluasi masing-masing, lalu memilih hasil terbaik.

Ketiga konsep di atas secara konsep tergolong kompleks, tetapi hasilnya menunjukkan bahwa pendekatan seperti ini tidak cocok untuk model berukuran kecil seperti Gemma-3-1B-IT. Dengan jumlah parameter yang terbatas, model tidak mampu menyimpan dan memproses konteks penalaran yang panjang sehingga membuat model tidak benar-benar melakukan penalaran dan

membuat model terpengaruh bias bawaannya yaitu memprediksi pola yang sering muncul atau memilih jawaban yang dianggap paling aman.

Hal ini dapat terlihat pada prompt Zero-Shot CoT yang menghasilkan performa terendah (0.403) karena model keterbatasan dalam memproses instruksi “think step by step” dan malah memberikan jawaban positive hampir di semua kasus (796 kali). Hal yang sama terjadi pada Few-Shot CoT, di mana model menjadi bingung karena mengikuti contoh yang banyak disertai dengan langkah-langkah penalaran yang diberikan. Akhirnya, model banyak mendominasi jawabannya dengan label neutral (501 kali).

Faktor kedua yang menyebabkan hasil macro F1-score atau kegagalan model dalam mengklasifikasikan label adalah adanya bias terhadap label positif yang memang merupakan bawaan model. Gemma-3-1B-IT merupakan model instruction-tuned assistant, yang secara alami cenderung memberi respons positif dan sopan. Bias ini terlihat jelas pada baseline teknik prompt yaitu Zero-Shot Prompting yang memprediksi label positive sebanyak 678 kali.

Dari hasil eksperimen dua teknik prompt berhasil mengurangi bias terhadap label positif, yaitu

- Role Prompting (skor tertinggi sebesar 0.564)

Dengan mendefinisikan peran model sebagai “kritikus restoran yang teliti dan sangat kritis” (“You are a meticulous and highly critical restaurant reviewer”), prompt ini mengubah sudut pandang model. Model tidak lagi menjawab dengan konteks yang ramah, tetapi model mulai menilai opini secara objektif. Hasilnya, prediksi positive turun ke 569 (terendah kedua) dan negative meningkat menjadi 327 (tertinggi di seluruh eksperimen).

- Generate Knowledge Prompting (skor tertinggi kedua sebesar 0.503)

Dengan meminta model menghasilkan satu kalimat pengetahuan yang relevan terlebih dahulu (“Soup is expected to be served hot.”), model terdorong untuk berpikir dengan mempertimbangkan fakta terlebih dahulu sebelum mengeluarkan label. Hal ini

menurunkan jumlah tebakan label positive menjadi 536 (terendah di antara semua teknik) dan meningkatkan keseimbangan antar kelas.

Kedua teknik ini menunjukkan bahwa menambahkan konteks yang mengubah perspektif berpikir model lebih efektif dibandingkan memaksa model untuk melakukan penalaran kompleks seperti CoT atau ToT.

Evaluasi menggunakan Macro F1-score menilai rata-rata F1 dari setiap kelas (positive, negative, neutral, dan conflict). Artinya, metrik ini lebih menekankan keseimbangan performa di semua kelas, bukan sekadar jumlah prediksi benar secara keseluruhan. Walaupun kelas positive paling banyak muncul dalam dataset, performa model justru ditentukan oleh kemampuannya mengenali kelas yang lebih jarang muncul, seperti negative, neutral, dan conflict. Jika model gagal mendeteksi salah satu kelas minoritas (misalnya conflict dengan skor $F1=0$), maka nilai Macro F1 akan langsung turun drastis. Hal inilah yang membuat Role Prompting menjadi teknik terbaik bukan hanya karena berhasil mengenali kelas neutral dengan baik, tetapi juga karena mampu memberikan hasil yang paling seimbang di seluruh kelas, dengan prediksi negative (327), neutral (100), dan bahkan conflict (4). Sebaliknya, teknik lain seperti Zero-Shot, Zero Shot + Chain of Thought, dan Emotion Prompting justru gagal dan sama sekali tidak dapat mengenali kelas conflict.

IV. Kesimpulan dan Saran

Berdasarkan hasil eksperimen dengan model decoder-only Gemma-3-1B-IT, dapat disimpulkan bahwa teknik prompting yang sederhana dan lebih fokus pada konteks justru memberikan hasil terbaik dibandingkan teknik yang membutuhkan penalaran rumit. Selain itu, Emotion Prompting yang menambahkan kalimat emosional (“This is very important...”) juga tidak memberi peningkatan terhadap kemampuan model mengenali kelas yang jarang muncul dibandingkan Zero-Shot Prompting. Teknik yang lebih langsung dan kontekstual seperti Role Prompting dan Generative Knowledge Prompting menghasilkan kinerja terbaik, dengan Role Prompting mencapai Macro F1 sebesar 0.56439. Kedua teknik ini mampu mengurangi kecenderungan model untuk selalu menjawab positif serta menjaga keseimbangan dalam mengenali kelas

minoritas seperti negative, neutral, dan conflict, yang berperan penting dalam peningkatan skor Macro F1-Score.

Untuk penelitian dan pengembangan selanjutnya, disarankan untuk mengoptimalkan strategi prompting sesuai kapasitas model. Kombinasi antara pendekatan berbasis konteks (Role Prompting) dan pengetahuan (Generative Knowledge Prompting) berpotensi menghasilkan prediksi yang lebih stabil dan akurat. Selain itu, penggunaan model dengan parameter lebih besar ($\geq 7B$) dapat dieksplorasi agar teknik penalaran seperti CoT dan ToT dapat bekerja lebih efektif.